

Lux: Prior Display-Server Architecture Specification

Visual Output Surface for AI Agents

May 2026 — v0.18.0

Contents

1	Overview	3
1.1	Scope	3
1.2	Companion Documents	3
1.3	Design Principles	3
2	System Architecture	4
2.1	Component Map	4
2.2	Data Flow	5
2.2.1	Write Path (Agent → Display)	5
2.2.2	Event Path (User → Agent)	5
2.2.3	Update Path (Incremental Patching)	5
3	Wire Protocol	6
3.1	Framing	6
3.2	Message Types	6
3.2.1	Client → Display	6
3.2.2	Display → Client	7
3.3	Element Kinds	7
3.4	Draw Commands	8
4	Display Server Internals	8
4.1	Frame Cycle	8
4.2	Multi-Scene State	9
4.2.1	Scene Lifecycle	9
4.2.2	Widget State Swap Pattern	10
4.3	Table Filtering	10
5	Frame Architecture	10
5.1	Frame Class	11
5.2	Frame Lifecycle	11
5.3	World Menu	11
5.4	Dock Bar	11
6	IPC and Process Management	12
6.1	Socket Discovery	12
6.2	PID Lifecycle	12
6.3	Auto-Spawn	12
6.4	Tool Reconnect Path	12
7	Performance	12
7.1	Frame Budget	12
7.2	Critical Path Analysis	13
7.3	Idle Screen	13
7.4	Memory	13
7.5	Known Limitations	13

8	Security	13
8.1	Threat Model	13
8.2	Render Function Security	14
8.3	Protocol Robustness	14
9	Platform Integration	14
9.1	macOS	14
9.2	Linux	15
10	MCP Tool Surface	15
11	Formal Model	16
12	Test Architecture	17

1 Overview

Lux is a visual output surface for Claude Code agents and local applications. In the display-server-centered architecture documented here, agents and apps push JSON element trees into a native ImGui window over Unix socket IPC. The system has four projection surfaces from a single codebase: Python library, CLI, MCP server, and Claude Code plugin.

The display server owns the render loop, scene state, widget state, interaction queue, and query surface. Agents are clients. All agent–display communication is message-passing over a framed JSON protocol on a Unix domain socket. The display server is single-threaded: no threads, no asyncio, no locks. It polls all I/O non-blocking each frame via `select()`.

1.1 Scope

This document is the canonical specification of Lux’s *prior display-server architecture*. It intentionally describes the display-server-centered design that shipped through the v0.18 line and does *not* treat the experimental Hub/domain slices now present in the repository as normative. Where the codebase contains newer transport or domain experiments, this document ignores them and records the stable design they grew out of.

1.2 Companion Documents

This document is the canonical, comprehensive description of the prior display-server system. The sibling files in `docs/architecture/` provide narrower supporting views:

- `display-server.tex` — Z specification of the core display-server state machine, including client lifecycle, scene replacement, patching, and interaction broadcast.
- `workspace-model.tex` — Z specification of frames, client identity, scene-to-frame routing, and World menu ownership.
- `claude-code-lux.tex` — Z specification of Lux as a consumer-facing visual surface inside the broader Claude Code workflow.
- `introspection-api.md` — the unified `QueryRequest/QueryResponse` envelope and the display introspection method catalog used by `inspect_scene()`, `list_scenes()`, and related tools. See Section §10.

Other architecture notes in this directory may discuss future-facing or experimental directions. They are intentionally out of scope for this specification.

1.3 Design Principles

- P1. Composable element tree.** Every element kind is small, nestable, and data-driven. If an agent can describe it as JSON, Lux renders it.
- P2. Protocol is the API surface.** The wire protocol is the only interface between agents and the display. No shared memory, no function calls, no Python imports required.
- P3. Single-threaded simplicity.** The display server uses a polling event loop with zero threads. All state is owned by the main thread. No synchronization primitives.
- P4. Incremental by default.** Scenes can be patched by element ID without replacing the entire tree. Table filters run at frame rate with zero MCP round-trips.
- P5. Multi-scene coexistence.** Each `show()` call opens a new tab. Scenes persist until explicitly dismissed or cleared. Same `scene_id` replaces content in-place.

2 System Architecture

2.1 Component Map

The Python package splits the prior display-server design across the protocol layer, the display/runtime layer, scene state management, and MCP/client adapters.

Module	File or package	Responsibility
Protocol	<code>src/punt_lux/protocol/</code>	Wire framing, the <code>FrameReader</code> streaming buffer, and the <code>message_to.dict</code> / <code>element_to.dict</code> dispatch helpers.
Elements	<code>src/punt_lux/protocol/elements/</code>	25 wire element dataclasses plus the <code>Patch</code> payload used by <code>UpdateMessage</code> . <code>JsonElementFactory</code> handles the ABC-backed <code>text</code> , <code>button</code> , and <code>dialog</code> path; <code>ElementCodec</code> covers the remaining kinds.
Messages	<code>src/punt_lux/protocol/messages/</code>	22 message dataclasses across the <code>scene</code> , <code>lifecycle</code> , <code>interaction</code> , <code>observer</code> , <code>menu</code> , and <code>introspect</code> families, with <code>MessageRegistry</code> as the wire dispatch table.
Display	<code>src/punt_lux/display/</code>	ImGui render loop, scene rendering, frame lifecycle, menu management, screenshot capture, and event flush. The core modules are <code>server.py</code> , <code>element_renderer.py</code> , <code>table_renderer.py</code> , <code>menu_manager.py</code> , <code>idle_screen.py</code> , and <code>texture_cache.py</code> .
Scene	<code>src/punt_lux/scene/</code>	Scene graph state. <code>manager.py</code> owns <code>SceneManager</code> , <code>frame.py</code> defines workspace frames, and <code>widget_state.py</code> holds per-scene ImGui control state.
Tools (MCP)	<code>src/punt_lux/tools/</code>	FastMCP tool surface. <code>server.py</code> owns the <code>FastMCP</code> instance and session lifecycle. <code>tools.py</code> defines display-facing helpers; <code>subscribe_tools.py</code> defines the separate session pub-sub helpers.
Socket server	<code>src/punt_lux/socket/_server.py</code>	<code>SocketServer</code> — Unix-domain socket lifecycle, non-blocking multiplexing of client connections, per-client <code>FrameReader</code> buffers, and send/recv to clients.
Query dispatcher	<code>src/punt_lux/query/_dispatcher.py</code>	<code>QueryDispatcher</code> — routes <code>QueryRequest</code> by method string, and owns the ring buffers for recent interaction events (max 200) and recent error log entries (max 100).
Client	<code>src/punt_lux/client.py</code>	Backward-compat shim that re-exports <code>DisplayClient</code> under the old name <code>LuxClient</code> .
Display client	<code>src/punt_lux/display/_client.py</code>	<code>DisplayClient</code> — typed <code>show</code> , <code>update</code> , <code>query</code> , <code>clear</code> , and <code>ping</code> methods, plus callback registration for <code>InteractionMessage</code> and <code>poll_event()</code> for queued <code>ObserverMessages</code> .
Gateway	<code>src/punt_lux/luxd.py</code>	Optional WebSocket gateway (<code>luxd</code>) and mcp-proxy entry point. In the prior architecture documented here it is transport and session bootstrap plumbing, not the source of truth for scene state or rendering.
Apps/Beads	<code>src/punt_lux/apps/beads.py</code>	Beads issue browser; loads <code>.beads/issues.jsonl</code> and renders a filterable table scene.
Show	<code>src/punt_lux/show.py</code>	<code>lux show</code> subcommand for pre-built scenes such as the beads board.
Paths	<code>src/punt_lux/paths.py</code>	<code>DisplayPaths</code> — socket discovery, PID lifecycle, and auto-spawn via <code>ensure()</code> . Module-level helpers also resolve the optional gateway's PID/port paths.
Runtime	<code>src/punt_lux/runtime.py</code>	<code>CodeExecutor</code> and <code>RenderContext</code> — compile and run agent-submitted render functions per-frame, gated by consent.

Module	File or package	Responsibility
Config	src/punt_lux/ config.py	<code>ConfigManager</code> reads and writes YAML frontmatter in <code>.punt-labs/lux.md</code> ; the live field today is the per-repo display-mode toggle (<code>display = y n</code>).
Hooks	src/punt_lux/ hooks.py	Claude Code lifecycle hook handlers (session start, post-bash).
Service	src/punt_lux/ service.py	<code>ServiceManager</code> — <code>launchd/systemd</code> lifecycle for <code>luxd</code> .
Remote	src/punt_lux/ remote.py	<code>ProxyConfigFile</code> — mcp-proxy WebSocket transport setup.
CLI	src/punt_lux/_\ main_\.py	Typer app with display, service, status, installation, and gateway management subcommands.

2.2 Data Flow

Three primary data paths connect agents to the display:

2.2.1 Write Path (Agent → Display)

1. Agent calls MCP tool (e.g., `show()`).
2. MCP server obtains a `DisplayClient` from the session's connection registry, then calls `client.show(scene_id, elements)`.
3. Client serializes `SceneMessage` as JSON, frames with 4-byte big-endian length prefix, sends over Unix socket.
4. `SocketServer` (`src/punt_lux/socket\_server.py`) receives the frame in `poll_clients()`, deserializes via `FrameReader.drain_typed()`, dispatches to `DisplayServer.handle_message()`.
5. `DisplayServer` delegates state updates to `SceneManager.handle_scene()` or `handle_framed_scene()`.
6. `DisplayServer._render_scene()` draws the active scene each frame via `ElementRenderer`.
7. `DisplayServer` sends an `AckMessage` back to the client.

2.2.2 Event Path (User → Agent)

1. User interacts with widget (click, slider drag, text input).
2. `ElementRenderer` or `TableRenderer` emits an `InteractionMessage` with `element_id`, `action`, `value`, and optional `scene_id` via the `emit_event` callback.
3. Message buffered in `DisplayServer._event_queue` and pushed to the `QueryDispatcher`'s ring buffer for later `list_recent_events()` queries.
4. Each frame, `_flush_events()` broadcasts all queued messages to all connected clients via `SocketServer.send_to_clients()` then clears the queue.
5. Client-side consumers handle those interactions through the display client, typically by registering `DisplayClient.on_event()` callbacks.

2.2.3 Update Path (Incremental Patching)

1. Agent calls `update(scene_id, patches)`.
2. `DisplayServer` receives `UpdateMessage`, delegates to `SceneManager.apply_update()`.
3. `_find_element()` (module helper in `scene/manager.py`) locates target by ID via depth-first search through the element tree.
4. Patch applied: `set` updates fields in-place, `remove` deletes element and cleans up widget state via `WidgetState.set(eid, None)` and `clear_suffix()`.

5. Routes to correct scene by `scene_id` (not limited to active tab); `SceneManager.resolve_scene()` looks up either the unfamed-scenes dict or the per-frame scene dict.

3 Wire Protocol

3.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----+
```

Maximum payload size: 16 MiB (2^{24} bytes). The `FrameReader` (in `src/punt_lux/protocol/\_\_init\_\_.py`) accumulates partial reads into a byte buffer and yields complete frames via `drain()` (raw dicts) or `drain_typed()` (Message dataclasses). Oversize frames raise `ValueError`, which `SocketServer` converts to a client disconnect.

3.2 Message Types

The message layer is split across the `scene`, `lifecycle`, `interaction`, `observer`, `menu`, and `introspect` families. In `src/punt_lux/protocol/messages/\_\_init\_\_.py`, the protocol surface is assembled into a `ClientMessage` union (12 request/command types), a `DisplayMessage` union (9 response/event types), and `UnknownMessage` for forward compatibility.

3.2.1 Client → Display

Type	Purpose
<code>SceneMessage</code>	Replace or add scene. Fields: <code>id</code> , <code>elements[]</code> , <code>title</code> , <code>layout</code> , <code>frame_id</code> , <code>frame_title</code> , <code>frame_size</code> , <code>frame_flags</code> , <code>frame_layout</code> .
<code>UpdateMessage</code>	Incremental patches. Fields: <code>scene_id</code> , <code>patches[]</code> (each: <code>id</code> , <code>set</code> , <code>remove</code>).
<code>ClearMessage</code>	Remove all scenes and reset state.
<code>PingMessage</code>	Heartbeat. Field: <code>ts</code> .
<code>ConnectMessage</code>	Client identity handshake. Field: <code>name</code> .
<code>MenuMessage</code>	Set agent-provided menu entries.
<code>ThemeMessage</code>	Apply theme by name.
<code>RegisterMenuMessage</code>	Register per-client menu items. Field: <code>items[]</code> .
<code>IntrospectRequest</code>	Request scene element tree (legacy ad-hoc op; retained alongside <code>QueryRequest</code> for backward compatibility). Field: <code>scene_id</code> .
<code>ListScenesRequest</code>	Request list of all active scenes (legacy ad-hoc op).
<code>ScreenshotRequest</code>	Request display screenshot (legacy ad-hoc op).
<code>QueryRequest</code>	Generic introspection/control request. Fields: <code>method</code> , <code>params</code> . The dispatcher routes by <code>method</code> string — see <code>introspection-api.md</code> .

SceneMessage frame fields (since v0.9.0): `SceneMessage` includes optional fields `frame_id`, `frame_title`, `frame_size` (initial `[width, height]`), `frame_flags` (dict of ImGui window flags such as `no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`), and `frame_layout` ("tab" or "stack").

3.2.2 Display → Client

Type	Purpose
ReadyMessage	Handshake on connect; protocol version, capabilities.
AckMessage	Acknowledges scene or update; <code>scene_id</code> , <code>ts</code> , optional <code>error</code> .
InteractionMessage	User interaction; <code>element_id</code> , <code>action</code> , <code>value</code> , <code>ts</code> , optional <code>scene_id</code> .
ObserverMessage	Session-scoped business event fan-out for the optional subscribe/publish path; <code>topic</code> + <code>payload</code> .
PongMessage	Ping response; round-trip latency probe.
IntrospectResponse	Scene element tree response.
ListScenesResponse	List of active scene summaries plus frame summaries (<code>scenes[]</code> , <code>frames[]</code>).
ScreenshotResponse	Path to captured PNG.
QueryResponse	Generic introspection/control response. Fields: <code>method</code> , <code>result</code> , optional <code>error</code> .

Unknown message types are deserialized as `UnknownMessage` for forward compatibility rather than disconnecting the client.

The dispatch table (`MessageRegistry` in `src/punt_lux/protocol/messages/registry.py`) is populated at import time by `register_codecs` calls from each family module. Tests can construct an isolated registry without touching the module-level default — this replaces the original `if/elif` chain in `message_from_dict` (PR #157).

3.3 Element Kinds

The wire layer exposes 25 element kinds plus the non-element `Patch` payload used by `UpdateMessage`. Three kinds (`text`, `button`, `dialog`) are decoded through `JsonElementFactory`; the remaining kinds are registered into `ElementCodec`. Each element is a frozen, slotted dataclass, and the `Element` union is assembled in `src/punt_lux/protocol/elements/\_\_init\_\_.py`.

Category	Element	Notes
basics	<code>text</code>	Content + style (heading, caption, success, error, code); optional <code>color</code> hex string
	<code>markdown</code>	CommonMark rendering via <code>imgui_md</code>
	<code>image</code>	File path or base64 data
	<code>separator</code>	Visual divider
	<code>progress</code>	Fraction bar with label
	<code>spinner</code>	Loading indicator
inputs	<code>button</code>	Label, action, disabled flag; <code>arrow</code> (directional) and <code>small</code> variants
	<code>slider</code>	Min/max/value, integer mode
	<code>checkbox</code>	Boolean toggle
	<code>combo</code>	Dropdown selection
	<code>input_text</code>	Single-line text input
	<code>radio</code>	Radio button group
	<code>input_number</code>	Numeric input with step buttons, min/max clamping, integer mode
	<code>color_picker</code>	RGB/RGBA hex picker; <code>alpha</code> (RGBA) and <code>picker</code> (full widget) modes
	<code>selectable</code>	Click-to-select item

Category	Element	Notes
dialog	dialog	Dialog root with title, children, handlers, and an internal model-backed lifecycle
layout	group	Row, column, or paged layout (<code>pages</code> + <code>page_source</code>)
	tab_bar	Tabbed container
	collapsing_header	Expandable section
	window	Movable, resizable floating panel
	tree	Recursive node hierarchy; <code>flat</code> flag for non-indented expansion
	modal	Modal popup dialog; container with children, emits "closed" event
graphics	draw	Canvas with typed draw commands (see §3.4)
	plot	ImPlot: line, scatter, bar series
table	table	Columns, rows, built-in filters, detail panel, pagination

3.4 Draw Commands

The `draw` element holds a tuple of typed `DrawCommand` records (PR #176, #177). Each command is a frozen, slotted dataclass that satisfies the `DrawCommand Protocol` (`TYPE: ClassVar[str], to_wire(self) -> dict, from_wire(cls, d, *, ctx) -> Self`).

Module	Record	Geometry
draw_commands_line	Line	Start, end, color, thickness.
	Polyline	Point list, color, thickness, closed flag.
draw_commands_shape	Rect	Top-left, bottom-right, color, thickness, rounding, filled.
	Circle	Center, radius, color, thickness, filled.
	Triangle	Three points, color, thickness, filled.
draw_commands_curve	BezierCubic	Four control points, color, thickness.
draw_commands_text	TextGlyph	Position, content, color.

Shared value classes live in `draw_values.py` and `draw_bounds.py`: `Point2` (validated $[x, y]$ pair), `Color` (`#RRGGBB` / `#RRGGBBAA` hex), `Thickness` (strictly positive stroke width), `Radius` (strictly positive), `Rounding` (non-negative). The `WireContext` in `draw_wire.py` carries the per-error prefix (`draw command [i] (kind)`) and the require-or-raise helpers (`require_number`, `require_sequence`, etc.). `DrawCommandDecoder` in `draw_decoder.py` dispatches on the wire `cmd` string to the appropriate `from_wire` classmethod; `DrawElement.commands` is typed `tuple[DrawCommand, ...]` and the renderer dispatches via a typed `match` statement.

4 Display Server Internals

4.1 Frame Cycle

The display server runs an ImGui render loop via `hello_imgui.run()`. Each frame, the `DisplayServer.on_frame()` callback executes four phases in sequence:

1. **Accept connections.** `SocketServer.accept_connections()` runs `select()` on the server socket with timeout 0. Accept if readable. Send `ReadyMessage` to the new client.
2. **Poll clients.** `SocketServer.poll_clients()` runs `select()` on all client sockets with timeout 0. Read available data into the per-client `FrameReader`. Dispatch complete messages to

`DisplayServer.handle_message()`, which routes by `isinstance` to the per-message-type handler. Remove errored sockets via `remove_client()`.

3. **Render scene.** Enable docking via `dock_space_over_viewport`. Render the idle flame background (`idle_screen.render_idle()`). Check for World menu background click. Render World panel if open. Render all workspace frames (see §5). Render the dock bar for minimized frames. Legacy fallback: if no frames exist, render unframed scenes as tabs.
4. **Flush events.** Broadcast all buffered `InteractionMessages` via `SocketServer.send_to_client()` to every connected client, then clear the queue. Each event is also recorded in `QueryDispatcher.recent_events` for later `list_recent_events()` queries.

All four phases are non-blocking. The frame completes in microseconds when idle. `ImGui` handles frame pacing via GLFW vsync; `hello_imgui.fps.idling.fps_idle = 30.0` caps the idle frame rate.

4.2 Multi-Scene State

Scene state is owned by `SceneManager` in `src/punt_lux/scene/manager.py`, exposed to the rendering layer through read-only properties on `DisplayServer.scene_manager`.

Field	Type and Purpose
<code>_scenes</code>	<code>dict[str, SceneMessage]</code> — unframed scenes; ordered by insertion; keyed by <code>scene_id</code> .
<code>_scene_order</code>	<code>list[str]</code> — explicit tab ordering for unframed scenes.
<code>_active_tab</code>	<code>str None</code> — currently selected tab in the unframed-scenes view.
<code>_scene.widget_state</code>	<code>dict[str, WidgetState]</code> — per-scene slider/checkbox/combo/text state.
<code>_dirty_windows</code>	<code>set[str]</code> — window element IDs needing initial position set.
Frame state	
<code>_frames</code>	<code>dict[str, Frame]</code> — all workspace frames, keyed by <code>frame_id</code> .
<code>_focus_frame_id</code>	<code>str None</code> — frame to auto-focus on next render.
<code>_scene_to_frame</code>	<code>dict[str, str]</code> — scene → frame lookup.
<code>_scene_to_owner</code>	<code>dict[str, int]</code> — scene → client fd ownership.
Display-server state	
<code>_event_queue</code>	<code>list[InteractionMessage]</code> on <code>DisplayServer</code> , drained each frame.
<code>_client_names</code>	On <code>SocketServer</code> , <code>fd → display name</code> from <code>ConnectMessage</code> .
<code>_menu_registrations</code>	On <code>MenuManager</code> , per-client agent menu items.

4.2.1 Scene Lifecycle

New scene (`scene_id` not in `_scenes`)

Appended to `_scenes` and `_scene_order`. Fresh `WidgetState` allocated. Becomes the active tab. Window elements marked dirty. Existing events preserved.

Replace-in-place (same `scene_id`)

Content replaced in `_scenes`. Widget state and render function state cleared for that scene. Events for elements that existed in the old scene but not the new scene are drained via `on_scene_replaced` callback (using full subtree ID collection through `_collect_ids()`). Events from other scenes are untouched.

Dismiss (tab close button)

Scene removed from all state dicts. Window element IDs removed from `_dirty_windows`. Active tab moves to the adjacent neighbor (next tab, or previous if rightmost was dismissed).

Clear (ClearMessage or menu)

All scenes, widget state, events, and dirty windows cleared. Fresh instances allocated (not `.clear()` on aliased objects).

4.2.2 Widget State Swap Pattern

The renderer’s per-kind `ElementRenderer`. `_RENDERERS` dispatch table all reads from `self._widget_state`. Rather than threading a `scene_id` parameter through every method, the display swaps the per-scene state dict into this instance variable before rendering each tab:

```
def _render_scene_tab(self, scene_id: str) -> None:
    self._element_renderer.widget_state = \
        self._scene_manager.scene_widget_state[scene_id]
    self._element_renderer.current_scene_id = scene_id
    scene = self._scene_manager.scenes[scene_id]
    for elem in scene.elements:
        self._element_renderer.render_element(elem)
```

This is the same pattern ImGui itself uses: “current context.” Zero changes required in any renderer method.

4.3 Table Filtering

Tables support built-in filtering at frame rate with zero MCP round-trips. `TableRenderer` (in `src/punt_lux/display/table/_renderer.py`) renders filter controls (search input, combo dropdowns) above the table and applies filters in the render loop:

1. Filter state stored in `WidgetState` with prefixed keys (e.g., `__tbl_search_{eid}`, `__tbl_filter_{col}_{eid}`).
2. `_apply_table_filters()` renders controls, snapshots state, detects changes.
3. `_filter_indexed_rows()` applies AND logic across all active filters (case-insensitive substring for search, exact match for combos).
4. Pagination resets to page 0 when filters change.
5. Detail panel renders a two-column metadata grid plus long-form body text for the selected row.

5 Frame Architecture

Since v0.9.0, scenes can target named *frames*—persistent ImGui windows that outlive client connections. Frames are the primary unit of workspace organization. The `Frame` class lives in `src/punt_lux/scene/frame.py`; `SceneManager` owns the `_frames` dict and the scene-to-frame routing.

5.1 Frame Class

Field	Type and Purpose
<code>frame_id</code>	<code>str</code> — Unique identifier.
<code>title</code>	<code>str</code> — Window title bar text.
<code>owner_fds</code>	<code>set[int]</code> — File descriptors of contributing clients.
<code>scenes</code>	<code>dict[str, SceneMessage]</code> — All scenes in this frame, keyed by <code>scene_id</code> .
<code>scene_order</code>	<code>list[str]</code> — Tab ordering.
<code>active_tab</code>	<code>str</code> <code>None</code> — Currently visible scene.
<code>minimized</code>	<code>bool</code> — Docked to bottom bar.
<code>cascade_index</code>	<code>int</code> — Stagger offset for initial positioning.
<code>initial_size</code>	<code>tuple[int,int]</code> <code>None</code> — Width/height hint (first use only).
<code>flags</code>	<code>dict[str,bool]</code> <code>None</code> — ImGui window flags (<code>no_resize</code> , <code>no_collapse</code> , <code>no_title_bar</code> , <code>no_background</code> , <code>no_scrollbar</code> , <code>auto_resize</code>).
<code>layout</code>	<code>Literal["tab", "stack"]</code> — Multi-scene composition mode.

Public read/write properties wrap each field (PR #159 extracted `_Frame` from the original `display.py` and renamed it `Frame`; constructor follows the PY-CC-1 `__new__` pattern).

5.2 Frame Lifecycle

Creation

A frame is created on the first `SceneMessage` with a `frame_id` not already in `_frames`. The client's fd is added to `owner_fds`.

Shared ownership

Multiple clients can contribute scenes to the same frame. Each client that sends a scene targeting the frame is added to `owner_fds`.

Orphan model

When a client disconnects, its scenes persist with an `_ORPHAN_FD` sentinel (`-1`) in place of the real fd. The frame remains visible and interactive. A new client connecting with the same `frame_id` adopts the orphaned scenes.

Dismissal

The close button removes the frame and all its scenes from state. Widget state for all scenes in the frame is cleaned up.

Focus management

New frames are positioned using cascade staggering (`cascade_index` increments per frame). Initial size is applied on first render only. `_focus_frame_id` triggers `SetNextWindowFocus` on the next render cycle.

5.3 World Menu

Right-clicking the docking background opens the World menu panel, providing frame management operations: list all frames, restore minimized frames, close all frames, and access display settings.

5.4 Dock Bar

Minimized frames appear as clickable buttons in a bottom dock bar. Clicking a dock bar button restores the frame to its previous position and size.

6 IPC and Process Management

The architecture documented here is centered on the display server process. Agents and apps connect to that display over a Unix socket, either directly or through higher-level client wrappers. Optional remote access uses `luxd` as a WebSocket/session gateway, but scene storage, widget state, interaction broadcast, and rendering still belong to the display process in this model.

6.1 Socket Discovery

`DisplayPaths` in `src/punt_lux/paths.py` resolves the display socket path in priority order:

1. `$LUX_SOCKET` environment variable.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.
3. `/tmp/lux-$USER/display.sock` (fallback).

The fallback path is chosen to stay within macOS's ~104-character `AF_UNIX` path limit.

The hub socket and PID files are resolved by module-level helpers in the same file: `hub_dir()`, `hub_pid_path()`, `hub_port_path()`, `hub_log_dir()`, `read_hub_port()`, `is_hub_running()`.

6.2 PID Lifecycle

- PID file written at `{socket_path}.pid` on server startup.
- Clients verify liveness via `os.kill(pid, 0)` before connecting.
- Stale socket cleanup: `DisplayPaths.cleanup_stale()` removes socket and PID file if the process is dead.

6.3 Auto-Spawn

`DisplayPaths.ensure()` spawns the display server if not running:

1. Checks socket existence and PID liveness.
2. If not running, spawns

```
python -m punt_lux display --socket {path}
with start_new_session=True
```

(detached from caller's process group).
3. Polls socket existence + PID liveness every 100 ms until ready or 5 s timeout.
4. Display logs to `{socket_path}.log`.

6.4 Tool Reconnect Path

The MCP surface wraps display-bound calls in `client_registry.with_reconnect()`: if an `OSError` (for example a broken pipe) occurs, the stale client is closed, a new connection is established, and the call is retried exactly once. There is no unbounded retry loop.

7 Performance

7.1 Frame Budget

Parameter	Value	Notes
Idle FPS	30	<code>fps_idling.fps_idle = 30.0</code>
Active FPS	60+	GLFW vsync, GPU-bound
Socket poll timeout	0	Non-blocking <code>select()</code>
Max message size	16 MiB	Hard limit in <code>FrameReader</code>
Connection timeout	5.0s	<code>ensure()</code> and handshake
Texture loading	Lazy	First render triggers PIL + OpenGL upload
Event ring buffer	200 entries	<code>QueryDispatcher._recent_events</code>
Error ring buffer	100 entries	<code>QueryDispatcher._recent_errors</code>

7.2 Critical Path Analysis

The frame-critical path is:

1. `select()` on N client sockets — $O(N)$, $N < 64$ in practice.
2. `FrameReader.feed()` — memcpy into buffer, yield frames. $O(B)$ where B is bytes received.
3. `DisplayServer.render_scene()` — ImGui draw list calls via `ElementRenderer.render_element()`. $O(E)$ where E is total element count across visible scene.
4. `_flush_events()` — send K events to N clients. $O(K \cdot N)$.

Table filtering is $O(R \cdot C)$ where R is row count and C is column count. This runs every frame but is fast for tables under $\sim 10,000$ rows. No benchmarks exist for larger tables; pagination (configurable page size) bounds visible row rendering.

7.3 Idle Screen

The idle screen renders ~ 60 draw calls per frame: 48 radial lines, 3 glow circles, 3 flame shapes (each ~ 7 bezier operations). This is negligible for a GPU-rendered immediate-mode UI at 30 FPS idle.

7.4 Memory

- **Scene storage.** Scenes are Python dataclass trees held in memory. No size limit beyond the 16 MiB message cap. Each scene is a separate dict entry; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs held in `TextureCache` until `_on_exit()`. No eviction policy. Image-heavy scenes accumulate GPU memory.
- **Widget state.** Per-scene `WidgetState` is a flat dict. Cleared on scene replacement. No growth concern.
- **Event queue.** Flushed every frame. Bounded by interaction rate (< 100 events/frame in practice).
- **Ring buffers.** `deque(maxlen=200)` for events and `deque(maxlen=100)` for errors; constant memory.

7.5 Known Limitations

1. **No ImGui ID scoping.** Widget IDs are derived from `element_id` alone, not scoped by scene. ImGui internal state (focus, open/closed) can bleed between tabs with colliding IDs.
2. **No texture eviction.** Long-running sessions with many images accumulate GPU memory.
3. **Filter is per-frame.** Table filtering re-executes every frame even when nothing changed. Not a problem at current scale but would benefit from dirty-flag optimization for 10,000+ row tables.

8 Security

8.1 Threat Model

The primary trust boundary is the Unix socket. The display server accepts connections from any process that can reach the socket file. In the default configuration (`/tmp/lux-$USER/`), the socket is protected by filesystem permissions (user-only directory, mode 0700).

Threat	Description	Mitigation
Socket hijacking	Another process connects to the socket and injects scenes or reads events.	Directory permissions (0700) on <code>/tmp/lux-\$USER/</code> .
Malformed messages	Client sends invalid JSON, unknown message type, or oversize payload.	Unknown types deserialized as <code>UnknownMessage</code> ; oversize payloads rejected by <code>FrameReader</code> .
Render function	Agent-submitted Python code executes arbitrary operations.	Consent dialog with AST warnings. User must explicitly click Allow.
Resource exhaustion	Scene with millions of elements or very large images.	16 MiB message cap. No element count limit (<code>ImGui</code> handles gracefully).

8.2 Render Function Security

Render functions are the only element kind that executes arbitrary code. The security model is **consent-based, not sandbox-based**:

1. **AST scanning**: a best-effort visitor flags suspicious imports (`os`, `subprocess`, `socket`, `ctypes`, `urllib`, etc.) and dangerous builtins (`eval`, `exec`, `open`, `__import__`). This is a *UX signal*, not a security boundary.
2. **Consent dialog**: an `ImGui` modal displays the full source code with line numbers and any AST warnings. The user must click “Allow” to proceed. Dismissing the modal (Esc) is treated as “Deny.” Consent is per-element: changing the source code resets consent.
3. **Execution**: `CodeExecutor` compiles the source and calls `render(ctx)` each frame. The `RenderContext` provides a persistent state dict, delta time, frame counter, canvas dimensions, and a `send()` callback for emitting events. Execution is **not sandboxed**: the code runs in the display server’s process with full access to the Python runtime. Runtime errors are caught and displayed, not propagated.

Implication: render functions should be treated as user-approved code, equivalent to running a script the user downloaded. The consent dialog provides informed consent, not isolation.

8.3 Protocol Robustness

- **Malformed JSON**: client disconnected, not crashed. `SocketServer` logs the error via the `on_error` callback and removes the client.
- **Unknown message type**: deserialized as `UnknownMessage` for forward compatibility.
- **Oversize frame**: `FrameReader` checks `payload_len` against `MAX_MESSAGE_SIZE` and raises; `SocketServer` converts to disconnect.
- **Partial reads**: `FrameReader` accumulates bytes across frames; no assumption of complete messages per `recv()`.
- **Identity fields protected**: `SceneManager.apply_update()` silently drops patches that attempt to change `id` or `kind` on an element.
- **Double-remove idempotent**: `SocketServer.remove_client()` is safe to call twice on the same socket.

9 Platform Integration

9.1 macOS

- **Dock presence**. The display server runs as a normal macOS Dock app per GLFW’s default activation policy (Regular). The Dock icon and Cmd-Tab entry are intentionally visible to aid operational debugging.

- **Process name.** `setproctitle("Lux")` makes the process identifiable in `ps` and `top`. Does not affect Activity Monitor’s binary name column.
- **Font discovery.** Prefers Arial Unicode or Helvetica; merges Apple Symbols for mathematical/Z notation glyphs and STIX Two Math for Mathematical Alphanumeric Symbols (U+1D400–1D7FF).
- **AF_UNIX path limit.** macOS limits Unix socket paths to ~104 characters. `tempfile.mkdtemp(prefix="lux-")` used in tests; production paths are short by design.

9.2 Linux

- **Font discovery.** DejaVu Sans or Noto Sans; merges Noto Sans Symbols2 and Noto Sans Math for Unicode coverage.
- **XDG compliance.** Prefers `$XDG_RUNTIME_DIR/lux/` for socket placement.
- **No Dock equivalent.** Window managers use the X11 window title (set to “Lux”).

10 MCP Tool Surface

The display-facing MCP surface is registered primarily in `src/punt_lux/tools/tools.py`. Simple scene and configuration tools are declared directly with `@mcp.tool()`; query-style operations route through the generic `QueryRequest` envelope where possible. Separate session-pub-sub tools live in `src/punt_lux/tools/subscribe\_tools.py` and are distinct from the display’s UI interaction stream.

Tool	Signature and Purpose
Scene management	
<code>show</code>	<code>(scene_id, elements, title?, layout?, frame_id?, frame_title?, frame_size?, frame_flags?, frame_layout?)</code> — Display a scene built from element dicts.
<code>show_table</code>	<code>(scene_id, columns, rows, filters?, detail?, flags?, title?, frame_id?, frame_title?)</code> — Convenience wrapper for a filterable table scene.
<code>show_dashboard</code>	<code>(scene_id, metrics?, charts?, table_columns?, table_rows?, title?, frame_id?, frame_title?)</code> — Convenience wrapper for metrics, charts, and a summary table.
<code>update</code>	<code>(scene_id, patches)</code> — Incremental JSON patches targeting elements by ID.
<code>clear</code>	<code>()</code> — Clear all scenes.
Display control	
<code>ping</code>	<code>()</code> — Latency probe; returns <code>PongMessage</code> .
<code>set_menu</code>	<code>(menus)</code> — Set agent-provided menu bar entries.
<code>register_tool</code>	<code>(label, tool_id, shortcut?, icon?)</code> — Register an item in the shared Lux Tools menu.
<code>set_theme</code>	<code>(theme)</code> — Apply display theme. Routes through <code>QueryRequest</code> .
Display configuration	
<code>set_window_settings</code>	<code>(opacity?, font_scale?, decorated?, fps_idle?)</code> — Configure display window appearance and idle frame rate.
<code>set_frame_state</code>	<code>(frame_id, minimized?)</code> — Control frame state (minimize/restore).

Tool	Signature and Purpose
Display introspection	
<code>inspect_scene</code>	(<code>scene_id</code>) — Element tree for a scene. The MCP helper routes through <code>QueryRequest</code> ; the legacy <code>IntrospectRequest</code> message remains in the wire protocol for compatibility.
<code>list_scenes</code>	() — All active scenes plus frame summaries. The MCP helper also routes through <code>QueryRequest</code> .
<code>screenshot</code>	() — Intended to capture the display as a PNG. The wire protocol retains a dedicated <code>ScreenshotRequest/ScreenshotResponse</code> path; the current MCP helper is transitional and still reaches the display through the query surface.
<code>get_display_info</code>	() — Display dimensions, frame count, client count, FPS, uptime, PID.
<code>get_window_settings</code>	() — Opacity, font scale, decorated, idle FPS.
<code>get_theme</code>	() — Current theme and available themes.
<code>list_clients</code>	() — Connected clients with display names and connect times.
<code>list_menus</code>	() — Registered menu items, per client.
<code>list_recent_events</code>	(<code>count?</code>) — Recent interaction events. Reads <code>QueryDispatcher._recent_events</code> (max 200).
<code>list_errors</code>	(<code>count?</code>) — Recent error log entries. Reads <code>QueryDispatcher._recent_errors</code> (max 100).
Per-repo config helpers	
<code>display_mode</code>	(<code>repo</code>) — Read the repo-local display mode from <code><repo>/punt-labs/lux.md</code> .
<code>set_display_mode</code>	(<code>mode</code> , <code>repo</code>) — Write the repo-local display mode and eagerly connect when enabling.
Session pub-sub helpers	
<code>subscribe</code>	(<code>topic</code>) — Subscribe the calling session to a topic within its own scope.
<code>unsubscribe</code>	(<code>topic</code>) — Remove the calling session's subscription to a topic.
<code>publish</code>	(<code>topic</code> , <code>payload?</code>) — Fan out a business event to subscribers in the calling session's scope.
<code>recv</code>	(<code>timeout?</code>) — Wait for the next session-scoped business event. This is <i>not</i> the display UI interaction stream.

The introspection surface is split between the generic `QueryRequest/QueryResponse` envelope and a small number of older dedicated request/response pairs kept for compatibility. At the protocol level, `screenshot` still expects the dedicated `ScreenshotRequest/ScreenshotResponse` path.

Six handlers are built into `QueryDispatcher` at construction time: `inspect_scene`, `list_scenes`, `list_clients`, `list_menus`, `list_recent_events`, `list_errors`.

Seven more handlers are registered by `DisplayServer` at construction time because they touch `ImGui` state: `screenshot` (stub — raises), `get_display_info`, `get_window_settings`, `get_theme`, `set_window_settings`, `set_frame_state`, `set_theme`.

11 Formal Model

The display server's state machine is formally specified in `docs/architecture/display-server.tex` using the Z notation (ISO 13568). The specification covers:

- Client connection/disconnection with `FrameReader` association.

- Scene replacement, incremental patching, and clearing.
- Interaction event generation and broadcast.
- FrameReader streaming buffer invariants.

The specification has been type-checked with Spivey’s `fuzz` and model-checked with ProB (7,943 reachable states explored). Refinement tests in `tests/test_display_refinement.py` verify commutativity between the abstract Z operations and the concrete Python implementation via an abstraction function (`tests/display_abstraction.py`).

Partition tests in `tests/test_display_partition.py` derive test cases from the Z specification using the Test Template Framework (TTF), covering all input space partitions of each operation schema.

Note: the formal model predates multi-scene tabs and models a single-scene display. The abstraction function maps multi-scene state to the active tab for backward compatibility. A specification update to model the full multi-scene state is pending.

12 Test Architecture

The test suite is broad and layered; the exact count changes regularly, so this document records the coverage shape rather than a brittle inventory snapshot.

- **Protocol and codec tests:** framing, serialization, typed draw-command decode, value validation, and query envelopes. Representative files: `test_protocol.py`, `test_draw_*`, `test_query.py`.
- **Display-state and manager tests:** pure state transitions for the display server without a live GUI. Representative files: `test_display_state.py`, `test_scene_manager.py`, `test_query_dispatcher.py`, `test_socket_server.py`.
- **Tooling and client tests:** `test_tools.py`, `test_display_client.py`, `test_show.py`, `test_cli.py`, `test_paths.py`, `test_service.py`, `test_remote.py`, and `test_config.py` cover the MCP surface, client wrappers, CLI, process helpers, remote config, and repo-local configuration.
- **Formal-model alignment:** `test_display_refinement.py` and `test_display_partition.py` tie the concrete Python implementation back to the Z specification.
- **Marker-gated integration and e2e tests:** `test_integration.py`, the integration-marked cases in `test_query.py`, and `test_e2e.py` exercise real sockets and, for the e2e path, a live display environment.

Quality gates are driven by the Makefile: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest` (default marker set), `check-oo` (OO ratchet against `.oo-baseline.json`), and `check-suppressions`. `make check` is the composite gate.